Adaptive Saliency Biased Loss for Object Detection in Aerial Images

Peng Sun, Guang Chen, and Yi Shang, *Senior Member, IEEE*

***Abstract*— Object detection in aerial images remains a challenging problem due to low image resolutions, complex backgrounds, and variations of sizes and orientations of objects in images. In recent years, several multiscale and rotated box-based deep neural networks (DNNs) have been proposed and have achieved promising results. In this article, a new method for designing loss function, called adaptive saliency biased loss (ASBL), is proposed for training DNN object detectors to achieve improved performance. ASBL can be implemented at the image level, which is called image-based ASBL, or at the anchor level, which is called anchor-based ASBL. The method computes saliency information of input images and anchors generated by DNN object detectors, weights of different training examples, and anchors differently based on their corresponding saliency measurements. It gives complex images and difficult targets more weights during training. In our experiments using two of the largest public benchmark data sets of aerial images, DOTA and NWPU VHR-10, the existing RetinaNet was trained using ASBL to generate an one-stage detector, ASBL-RetinaNet. ASBL-RetinaNet significantly outperformed the original RetinaNet by 3.61 and 12.5 mean average precision (mAP), respectively, on the two data sets. In addition, ASBL-RetinaNet outperformed ten other state-of-the-art object detection methods.**

***Index Terms*— Adaptive saliency bias loss, aerial image, object detection, Retinanet.**

I. Introduction

In RECENT years, deep neural networks (DNNs) have achieved huge success in many areas of computer vision, such as image classification [1]–[3], object detection [4]–[11], and remote sensing [12]–[15]. For object detection, although significantly advances have been made, detecting small objects remains a challenging problem. For instance, convolutional neural networks (CNNs) have been applied to image classification problems in ImageNet [16] and have surpassed the error rate of human vision ability; however, the best-performing object detection model on the COCO data set [17] only achieved around 40 mean average precision (mAP) when the Intersection over Union (IoU) of the ground truth box and predicted box is 0.5.

Manuscript received July 28, 2019; revised November 11, 2019 and February 16, 2020; accepted February 29, 2020. Date of publication March 25, 2020; date of current version September 25, 2020. This work was supported in part by a grant from the Missouri Department of Conservation. *(Corresponding author: Yi Shang.)*

Peng Sun is with The Climate Corporation, San Francisco, CA 94103 USA (e-mail: ps793@mail.missouri.edu).

Guang Chen and Yi Shang are with the Department of Electrical Engineering and Computer Science, University of Missouri, Columbia, MO 65201 USA (e-mail: gcgrf@mail.missouri.edu; shangy@missouri.edu).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TGRS.2020.2980023



Fig. 1. Four sample images from the DOTA data set, showing target objects (boat, truck, car, airplane, and harbor) of a wide range of sizes and orientations in aerial images. Harbor, plane, SV, and LV are the label classes of the target objects.

Existing DNNs for object detection in computer vision can be classified into one-stage or two-stage detectors. A two-stage detector consists of a detection network to generate region proposals, followed by a classification network to recognize the object in each proposed region. In other words, in the first stage, a DNN, such as regions with CNN features (RCNN) [4], is used to generate the candidate location of each target object; then, in the second stage, another DNN determines whether each candidate location contains a target object or not. In comparison, an one-stage detector employs a single DNN to perform localization and recognition in one shot. Examples of one-stage detectors include YOLO [8], single shot multibox detector (SSD) [11], and RetinaNet [18]. One-stage detectors usually run faster than two-stage detectors, while achieving similar or better accuracy. For example, one-stage detector RetinaNet [18] outperformed one of the best two-stage detectors, Faster RCNN [5], with a 4.0-mAP improvement on the COCO data set [17].

Recently, deep learning has been applied to various types of sensor data domains, such as bio-sensors [19], [20] and remote sensing [12]–[15], [21]. Detecting objects in aerial images, as shown in Fig. 1, is different from detecting objects in conventional images in several ways: 1) objects in aerial images often appear with arbitrary orientations; 2) the size variations of objects in aerial images are much larger than

0196-2892 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

those in conventional images; 3) many small objects may be packed together in aerial images; and 4) some aerial images have uniform and simple backgrounds, while others have complex backgrounds. These characteristics make object detection in aerial images challenging. To improve detection accuracy, new techniques, such as rotated-box-based [22], [23] and multiscale-based DNNs [22], have been proposed. However, these networks are complicated with a large number of parameters. They have very slow inference speed, much slower than one-stage detectors, such as YOLO, SSD, and RetinaNet, which can achieve real-time detection.

In this article, we propose a new loss objective function, adaptive saliency biased loss (ASBL) that can be used to train existing one-stage detectors to achieve higher detection accuracy, while keeping the one-stage detectors' speed advantage. We use the techniques of saliency-based detection [24]–[26] in DNNs to compute different levels of features in aerial imagery and extract useful saliency information with respect to input images and target objects. The new loss function has two forms, image-based loss and anchor-based loss. In the image-based loss function, input images are weighted differently based on the overall saliency information in an image, with more complex images (higher saliency values) getting more weights during training. In the anchor-based loss function, anchors generated by the detector networks are given adaptive weights, with more likely object regions (higher saliency values) getting more weights. The goal of this new loss function is to focus training on more complex images and more likely target areas, which prevents the vast number of easy images and negative anchors from dominating the cross-entropy loss of the neural network model. The loss function can be applied to any one-stage multiscale feature extraction detector network. In our experiment, the loss function was applied to train RetinaNet [18] and the trained network is called ASBL-RetinaNet. Two large widely used public benchmark data sets of aerial images were used for performance evaluation: DOTA [21] and NWPH VHR-10 [27]. Experimental results show that ASBL-RetinaNet significantly outperformed the other state-of-the-art object detectors. In terms of object detection in remote sensing, previously, we proposed the similar work using saliency information to improve the performance of DNN [28], the main difference compared with the previous work is that this article has significant new materials, including the anchor-based ASBL and more comprehensive experimental results on DOTA and NWPU VHR-10 data sets.

The rest of this article is organized as follows. Section II introduces the related work of object detection, with an emphasis on the techniques related to object detection in aerial images. Section III presents the theory of the ASBL function. Section IV presents an ablation study and experimental results on benchmark data sets DOTA and NWPU VHR-10. Finally, Section V summarizes this article.

II. Related Work

For detecting objects in aerial images, there are four major approaches: template matching-based approach, knowledge-based approach, object-based image analysis (OBIA)-based approach, and machine learning-based approach, as discussed in [29]. Machine learning-based methods, such as the histogram of oriented gradients (HOG), Haar-like, and SR-based information, are first extracted, and feature fusion and dimension reduction are used to extract useful information. The features extracted are then fed into classifiers, such as support vector machine (SVM) and Adaboost. In recent years, deep learning techniques have been applied to object recognition in images and achieved state-of-the-art results. Most existing works on object detection using deep learning are for natural images taking on the ground. Due to the unique features of aerial images that are different from the natural images, these object detectors did not perform very well on aerial images. Researchers [13]–[15] have proposed various methods based on fine-tuning pretrained network models, such as pretrained on ImageNet [16] and COCO data [17], to get improved performance on aerial images. In addition, several new techniques have been proposed to address the unique features of aerial images directly, including multiscale and multiangle detection techniques, to achieve improved performance. For example, Cheng *et al.* [30], [31] proposed rotation-invariant deep learning models with regularization to achieve state-of-the-art performance on remote sensing images. Moreover, weakly supervised learning methods [32] have been proposed to learn features of different levels in an unsupervised manner to capture the structural information of objects in remote sensing images.

For object detection in general images, DNN models, such as SSD [11], YOLO [8], and RetinaNet [18], have been proposed and have achieved good performance. In the beginning, one-stage detectors had faster inference speeds than two-stage detectors but their detection accuracy is generally not so good. However, the new one-stage detector RetinaNet [18] was able to outperform state-of-the-art two-stage models on both speed and accuracy: 4.0 mAP higher than Faster RCNN [5] on the COCO data [17]. RetinaNet [18] combines the advantages of the SSD [11] and YOLO [8] networks by performing a multilayer feature extraction and then information fusion through a subnetwork. RetinaNet [18] uses focal loss to address a key problem of one-stage detectors—the extreme imbalance between foreground and background classes during training.

In training object detectors, the imbalance of easy and hard examples and the imbalance of positive and negative examples will lead to poor performance. In general, more hard positive examples enable the neural network model to discover and expand sparsely sampled minority class boundaries, while more hard negative examples improve the margins of minority class boundaries corrupted by visually similar classes. Random sampling techniques have been used to address the class imbalance problem [33]. Mining hard examples has been shown to be effective [34]. Recently, the online hard example mining (OHEM) method [10] has been proposed, which is an online bootstrapping algorithm for training region-based ConvNet object detectors like Fast RCNN [35]. For one-stage detector, specifically SSD [11], the ratio of positive and negative examples with random sampling is more balanced, which led to faster convergence and more stable training. However, most one-stage detectors still have the problem of unbalanced positive and negative anchors. For example,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

deconvolutional single shot detector (DSSD) [36] and RetinaNet could have up to 40k and 100k anchors on benchmark images, with a very small fraction of positives. The proposed new loss function aims to address both the training example imbalance and anchor imbalance problem.

The focal loss function [18] was designed to improve the cross entropy loss function on class-imbalance and easy/hard example imbalance problem in neural network training. The cross entropy between a predication by a network model and the target label is defined as follows:

$$\mathrm{CE}(p, y) = \begin{cases} -\log(p), & \text{if } y = 1 \\ -\log(1-p), & \text{if } y = 0 \end{cases} \tag{1}$$

where p is the class probability produced by the model, and y is the ground-truth class label. $y = 1$ and $y = 0$ mean positive and negative classes, respectively. For convenience, let

$$\begin{aligned} \mathrm{CE}(p, y) &= -\log(p_t) \\ p_t &= \begin{cases} p, & \text{if } y = 1 \\ 1-p, & \text{if } y = 0. \end{cases} \end{aligned} \tag{2}$$

Usually, after an object detector, DNN is trained based on cross-entropy, easy examples still incur a small amount of loss. When the number of easy examples is very large compared to hard training examples, the sum of the small losses of the easy examples can be large, dominating the loss of hard examples. The focal loss function prevents easy examples from dominating the loss function and the gradient. It introduces a weight factor α for foreground and $1-\alpha$ for background of an image (α_t to represent it), and add a new factor $(1-p_t)^\gamma$ to cross-entropy loss with a control parameter γ. Now, if an example is misclassified, the new factor will be near 1 and the loss function will be about the same; however, if an example is predicted correctly, the factor will scale the loss to a small value, toward 0, so that the impact of the easy examples in the loss function will become very small. With the control parameter γ, the scale of the factor can be tuned empirically. Overall, the focal loss function is

$$\mathrm{FL}(p, y) = \alpha_t * (1-p_t)^\gamma * \mathrm{CE}(p, y) \tag{3}$$

where γ and α_t are constant hyper-parameters. We used $\gamma = 2$ and $\alpha = 0.25$ in our experiments, as suggested in previous work for natural image object detection.

III. Adaptive Saliency Biased Loss Functions

When a detector network is trained using a set of training images, the training examples are commonly treated equally. If the majority of the training examples are easy cases, the trained model may focus on the easy cases, and the hard cases do not exert sufficient influence to make the model more generalizable and work well on hard cases. In addition, because the predictions of one-stage object detectors are based on anchors of reception fields, class imbalance, and improper hyper-parameter selection could lead to poor performance. To address these issues, we propose a new loss function, called ASBL, to train and improve one-stage object detectors. The loss function has two forms, one giving complex images more weights during training and the other giving better anchors more weights. The idea is to use the saliency information of whole images and anchor regions to represent the complexity and important areas of input images and dynamically weight input images and anchors in different layers of feature maps during training.

A. Image-Based ASBL Function

We propose an image-based ASBL function to direct training to focus more on hard training examples, that is, images containing objects that are hard to detect. Some existing methods use Edgebox [37] to determine the complexity of training images, such as WiderFace [38] and other data sets [39], [40]. However, these methods are complicated and the proper parameter values in Edgebox are hard to decide.

In our method, a pretrained DNN is used to compute the complexity of input images based on the assumption that an input image is more complex if there are more activated neurons in a hidden layer. Many state-of-the-art DNNs have been trained on large-scale image data sets, such as ImageNet [16], and have the ability to extract features of general objects at different levels. In computer vision, a saliency map of an image is a value array representing each pixel's importance.

In our implementation, we use pretrained DNNs by ImageNet as saliency estimators to estimate the complexity of an input image. Specifically, we use a CNN pretrained on ImageNet as a saliency estimator and extract features from different convolutional layers to represent the complexity of an input image, S_I, as defined in the following formula:

$$S_I(x) = \frac{1}{C * W * H} \sum_{c=1}^{C} \sum_{w=1}^{W} \sum_{h=1}^{H} f_{c,w,h}(x) \tag{4}$$

where S_I is the saliency of an image defined as the average activation of a convolutional layer; x is the input image; $f_{c,w,h}$ is the output of a certain convolutional layer in a pretrained CNN with output dimension $C * W * H$. According to this formula, easy images would have fewer activated neurons in a convolutional level and will result in smaller values of S_I than complex images. S_I values computed based on $f_{c,w,h}$ values from different convolutional layers in a DNN represent complexity at different feature levels. In the experiments, we investigated S_I computed from different individual convolutional layers, as well as composite S_I from multiple layers, which captures a multiscale view for each input image.

To bound the range of S_I, we propose the following normalization:

$$S'_I(x) = \frac{S_I(x) - S_{\min}}{S_{\max} - S_{\min}} (S_{\mathrm{ub}} - S_{\mathrm{lb}}) + S_{\mathrm{lb}} \tag{5}$$

where S_I is the original saliency value; $S_{\min}$ and $S_{\max}$ are the overall minimum and maximum S_I value of the training set, respectively, calculated once at the beginning of the training phase; S_{ub} and S_{lb} are constant upper bound and lower bound set as hyper-parameters of the method. In our implementation, we set S_{ub} to 1 and tried different values of S_{lb}, such as 0.3, 0.5, and 0.7.

The new Image-Based ASBL function, ASBL_I, combines saliency information with focal loss as follows:

$$\mathrm{ASBL}_I(x, p, y) = S'_I(x) * \mathrm{FL}(p, y) \tag{6}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

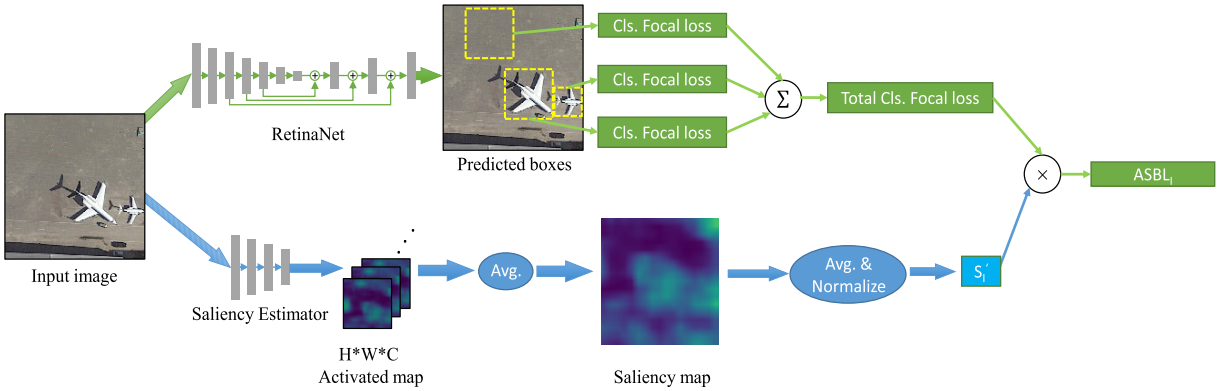


Fig. 2. Framework of computing the image-based ASBL (ASBL$_I$) function based on RetinaNet and ResNet50. The top branch of the framework consists of the original RetinaNet and the focal loss computed based on its predictions. The bottom branch consists of a saliency estimator network (e.g., ResNet50), and the S'_I value computed based on the feature maps from a certain hidden layer of the estimator, such as the activation of the conv2 layer of ResNet50.

where x is the input image; p is the class probability generated by the neural network detector; y the ground-truth class label; and FL(p, y) the focal loss in (3). The saliency value of each image becomes the weight on the focal loss of the image. Therefore, the loss values from easy cases will become smaller due to smaller S'_I values.

Fig. 2 shows the framework of computing ASBL$_I$ based on RetinaNet and ResNet50. The top branch of the framework consists of the original RetinaNet and the focal loss computed based on its predictions. The bottom branch consists of ResNet50, which is used as the saliency estimator network, and the S'_I value computed based on its feature map at a certain hidden layer, such as the activation of conv2 layer of ResNet50. ASBL$_I$ is generated by multiplying focal loss with S'_I. After ASBL$_I$ is computed for each training image, an existing training algorithm, such as stochastic gradient descent (SGD) or ADAM solver, can be used to train the RetinaNet. After training, the RetinaNet can be used to detect objects in input images. Note that the RetinaNet trained using ASBL$_I$ has exactly the same network architecture as the original RetinaNet, but different weights.

The framework shown in Fig. 2 is general and not restricted to RetinaNet and ResNet50. Other object detector networks can be used in place of RetinaNet and other image classifier or detector networks can be used in place of ResNet50 as the saliency estimator.

Image-based ASBL has two major advantages.

1) As the loss converges in training, the hard training examples will contribute more and the easy examples will contribute less to the trained model because the easy cases will have smaller loss values.
2) When S'_I are computed based on features extracted from different convolution layers in the saliency estimator network, multiscale features are incorporated into the overall loss function. For instance, the lower level features represent smaller objects more than larger objects.

B. Anchor-Based ASBL Function

One-stage detectors, such as RetinaNet and DSSD, use anchors, also called anchor boxes, which are a set of predefined bounding boxes of a certain height and width, to capture the scale and aspect ratio of specific object classes to be detected. A set of anchors with different scale and aspect ratios are created for every point (centered at this point) in a feature map of a one-stage detector. During detection, the predefined anchor boxes are tiled across the image. To fully cover an input image, one-stage detectors usually generate many anchors of different sizes. Each anchor in a multiscale feature map generates a prediction for the category and location of objects. Since the amount of foreground in images is much less than the background, a large number of negative anchors may be generated for the background compared to the number of positive anchors for foreground objects. A large number of redundant negative anchors cause unbalanced classification problems in one-stage object detectors. In aerial images, most objects of interest are small, which leads to more redundant negative anchors than those for larger objects. For instance, in a 256 * 256 feature map of the Conv2 layer in ResNet50, there are 256 * 256 * 9 anchors (three different ratios and three different scales of anchors for each point in the feature map) in total. A small object of size 5 * 5 in a feature map is covered by 5 * 5 * 9 positive anchors, while other anchors are negative. On the other hand, a larger object of size 10 * 10 in a feature map is covered by 10 * 10 * 9 anchors. The number of redundant negative anchors of larger objects is much smaller than those of smaller objects.

To address this issue, we propose a new anchor-based ASBL function, called ASBL$_A$. We first use Bayesian theory to calculate the probability of saliency of each point in a feature map and then use these values to compute the saliency of the anchors centered at each point. Let I denote an input image, S for saliency values in the range [0, 1], A for anchors. $Pr(S|A, I)$, the posterior probability of saliency values of anchors on each point in a feature map is computed as follows:

$$Pr(S|A, I) \propto Pr(A|S, I) * Pr(S|I) \tag{7}$$

where $Pr(S|I)$ is the prior probability of saliency for each point in a feature map given an input image I; $Pr(A|S, I)$ is the likelihood probability of positive anchors on each point given saliency S and input image I.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

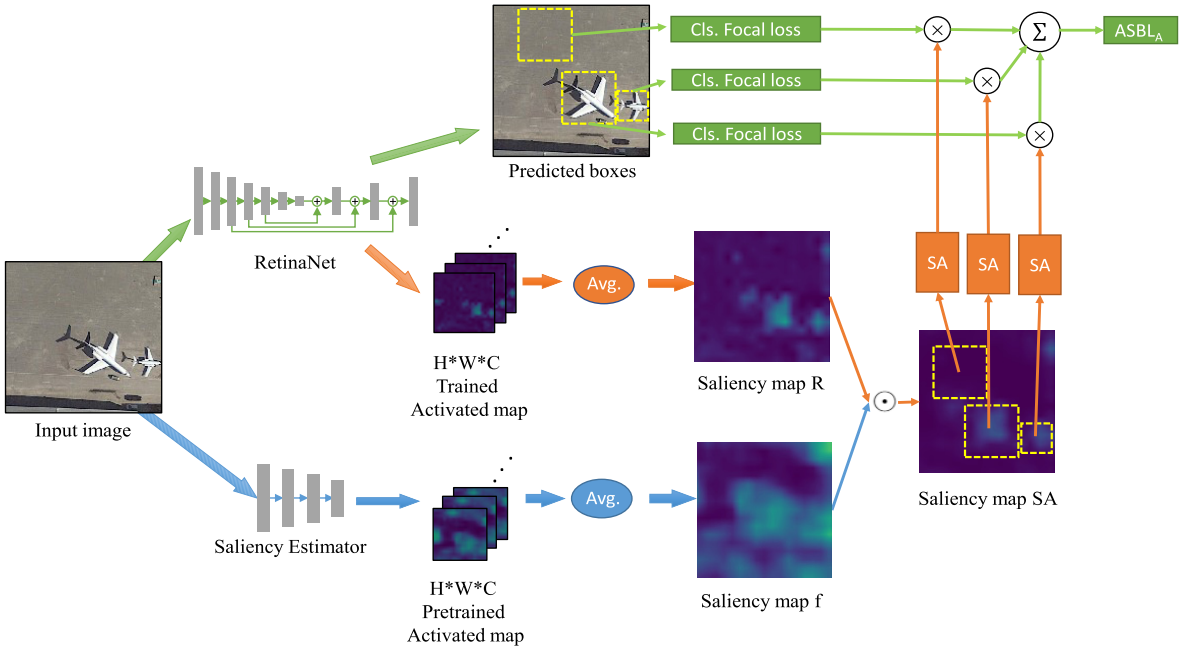


Fig. 3. Framework of computing the anchor-based ASBL function, ASBL_A based on RetinaNet and ResNet50. The top branch consists of the original RetinaNet and the focal loss computed based on its predictions. The middle branch computes the saliency map $R_{c,u,v}$ based on features maps extracted from RetinaNet. The bottom branch consists of an image saliency estimator network (e.g., ResNet50) and the saliency map $f_{c,u,v}$ computed based on the feature maps from a certain hidden layer, such as the activation of conv2 layer of ResNet50. Each value in $\text{SA}_{u,v}$ is used to weight the classification loss of anchors.

The framework of computing the Image-Based ASBL (ASBL_I) function based on RetinaNet and ResNet50. The top branch of the framework consists of the original RetinaNet and the focal loss computed based on its predictions. The bottom branch consists of ResNet50, which is used as the saliency estimator network, and the S'_I value computed based on the feature maps from a certain hidden layer, such as the activation of conv2 layer of ResNet50.

In our implementation, feature maps generated by an one stage object detector, such as RetinaNet, is used as the approximation of $Pr(A|S, I)$. $Pr(S|I)$ is derived from an object detector, such as ResNet50, in a way similar to the saliency estimator in ASBL_I. The anchor saliency map of an input image x is computed as follows:

$$\text{SA}_{u,v}(x) \propto \frac{1}{C}\sum_{c=1}^{C} R_{c,u,v}(x) \odot \frac{1}{C}\sum_{c=1}^{C} f_{c,u,v}(x) \tag{8}$$

where u and v are the coordinates of a point in the anchor saliency map, which is of the same size of feature maps; C is the number of feature maps (or channels); $R_{c,u,v}$ is a feature map generated by an one-stage object detector; $f_{c,u,v}$ is a feature map from a convolution layer of a pretrained image detector, which is of the same size as $R_{c,u,v}$; and $\text{SA}_{u,v}$ is the saliency value for each set of anchors. We average all the channels of $R_{c,u,v}$ and $f_{c,u,v}$ to get the likelihood probability of positive anchors of each point in the feature maps and the prior probability of saliency for the same point. $f_{c,u,v}$ is used to estimate the prior knowledge of the complexity of an image and $R_{c,u,v}$ is the likelihood of positive anchors of objects in an image. During training, $R_{c,u,v}$ is dynamically updated and learned so that $\text{SA}_{u,v}$ is also dynamically updated.

Finally, the anchor-based ASBL, ASBL_A, is computed as follows:

$$\text{ASBL}_A(x, p, y) = \sum_{a=1}^{As}\sum_{v=1}^{H}\sum_{u=1}^{W} \text{SA}_{u,v}(x) * \text{FL}_{u,v,a}(p, y) \tag{9}$$

where $\text{FL}_{u,v,a}(p, y)$ is the loss function for each anchor; As is the number of anchors for a feature map; W and H are the dimensions of each feature map.

Fig. 3 shows the framework of computing the Anchor Based ASBL Function, ASBL_A based on RetinaNet and ResNet50. The top branch consists of the original RetinaNet and the focal loss computed based on its predictions. The middle branch computes the saliency map $R_{c,u,v}$ based on features maps extracted from RetinaNet. The bottom branch consists of an image saliency estimator network (e.g., ResNet50) and the saliency map $f_{c,u,v}$ computed based on the feature maps from a certain hidden layer, such as the activation of conv2 layer of ResNet50. Each value in $\text{SA}_{u,v}$ is used to weight the classification loss of anchors.

ASBL_A has these three properties.

1) $R_{c,u,v}$ will be dynamically updated during the training process so that $\text{SA}_{u,v}$ will be learned.
2) Each set of anchors of each point in the feature maps have the same weighting value. Anchors predicted wrong will carry more weights.
3) Smaller $\text{SA}_{u,v}$ means the content in the anchor is simple, which leads to a small contribution in the overall loss function.

During the training of RetinaNet, the feature map generated by a one-stage object detector, $R_{c,u,v}$, will contain more and more accurate features of target objects, similar to the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

attention mechanism, so that the anchor saliency map, $SA_{u,v}$, contains more accurate information for each anchor as the training goes. During training using $ASBL_A$, anchor saliency map, $SA_{u,v}$, will be updated to contain more information of the target objects while information of redundant negative anchors without useful information is ignored. In our research, we visualized saliency maps, such as the ones shown in Fig. 3, and found that the weights of the negative anchors in the loss functions did reduce during training. Thus, the training will focus more on the anchors with higher saliency information, which corresponds to the target objects.

C. ASBL-RetinaNet

The two new loss functions, $ASBL_I$ and $ASBL_A$, can be used independently to train existing one-stage object detector. They can also be used together in a single training process to achieve further improvement. Based on our experimental results, we propose the ASBL strategy as follows: For a given number of training epochs, use $ASBL_I$ in the first half of the training process and $ASBL_A$ in the second half.

ASBL can be implemented based on any existing one-stage DNN detector. For example, if ASBL uses RetinaNet [18], which is one of the best one-stage detectors, as shown in Figs. 2 and 3, we call the method ASBL-Retina. In this case, the detector DNN model is RetinaNet, while the training is based on the ASBL loss function. The performance of the trained network can be compared directly with that of RetinaNet trained based on the original loss function. The inference times of the networks trained in the two different ways will be the same.

In our implementation, ResNet50 [3] is used as the saliency estimator. In order to extract the same level of features as RetinaNet, we pretrained ResNet50 using ImageNet with two more convolution blocks to obtain intermediate results in the same dimension and shape as the encoder part of RetinaNet. The features extracted from the revised ResNet50 are denoted as {C2-C7}. The corresponding feature maps in the encoder part of RetinaNet are denoted as {P2-P7}. These features are used to generate saliency information of input images. Each extracted feature can be used as a weight factor of the training images in the loss function.

Algorithm 1 presents the method of training RetinaNet using ASBL. The inputs are the RetinaNet and ResNet50 DNN models. ResNet50 provides stationary image level saliency information. The updated parameters, W, of RetinaNet is the output. First, we train ResNet50 with two more convolution blocks with the same architecture of encoder of RetinaNet using ImageNet in order to generate image level saliency information. Then, we train RetinaNet for 50 epochs: the first 25 epochs based on $ASBL_I$, and the last 25 epochs based on $ASBL_A$. In the first 25 epochs, $ASBL_I$ is calculated by ResNet50 using (4) to (6). In the last 25 epochs, $ASBL_A$ is computed based on the feature maps of ResNet50 and RetinaNet according to (8) and (9). The total number of training epochs is a hyper-parameter and is set to 50 in our experiments based on empirical results.

Algorithm 1 Training ASBL-RetinaNet

Require: RetinaNet, ResNet50
Ensure: Parameter W in RetinaNet
  $nEpoch = 50$
  $N = 0$
  Train ResNet50 with 2 more conv-block using ImageNet
  while $N < nEpoch$ **do**
    if $N < nEpoch/2$ **then**
      1. Calculate S'_I using C2 of ResNet50
      2. Train RetinaNet based on $ASBL_I$
    else $\{N \geq nEpoch/2\}$
      1. Output C3-C7 feature maps of ResNet50
      2. Output P3-P7 feature maps of RetinaNet
      3. Calculate $SA_{u,v}$
      4. Train RetinaNet based on $ASBL_A$
    end if
    $N = N + 1$
  end while

IV. Experimental Results

In this section, experimental results on two large benchmark data sets of aerial images, DOTA and NWPU VHR-10, are presented.

A. Data Set

DOTA is the largest and most diverse public data set for multiclass object detection in aerial images. It consists of 2806 images captured by different cameras. The images were acquired from Google Earth and the China Center for Resources Satellite Data and Application. It has 15 object categories: plane, baseball diamond (BD), bridge, ground field track (GTF), small vehicle (SV), large vehicle (LV), tennis court (TC), basketball court (BC), storage tank (SC), soccer ball field (SBF), roundabout (RA), swimming pool (SP), helicopter (HC), and harbor. Across these categories, 57% are small objects that are within 50 * 50 pixels. The DOTA data set is split into training (1/2), validation (1/6), and test (1/3) sets.

NWPU VHR-10 is another widely used public data set that consists of a positive image set including 650 images and a negative image set including 150 images over ten object categories. In our experiments, we used the official 1172 images (400 * 400 pixels) cropped from the positive image set of NWPU VHR-10 [27]. The data set contains ten classes of geo-spatial objects: airplane, ship, storage tank, BD, TC, BC, GTF, harbor, bridge, and vehicle. To have a fair comparison with previous results, we used the existing train, validation, and test data set that contains 679 images for training, 200 images for validation, and the rest 293 images for the test. For performance evaluation, we followed the official way [27] to evaluate the performance of our methods.

B. Evaluation

The performance metric used in our experiments is mAP, as for PASCAL VOC [41]. In our experiments, we focused

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

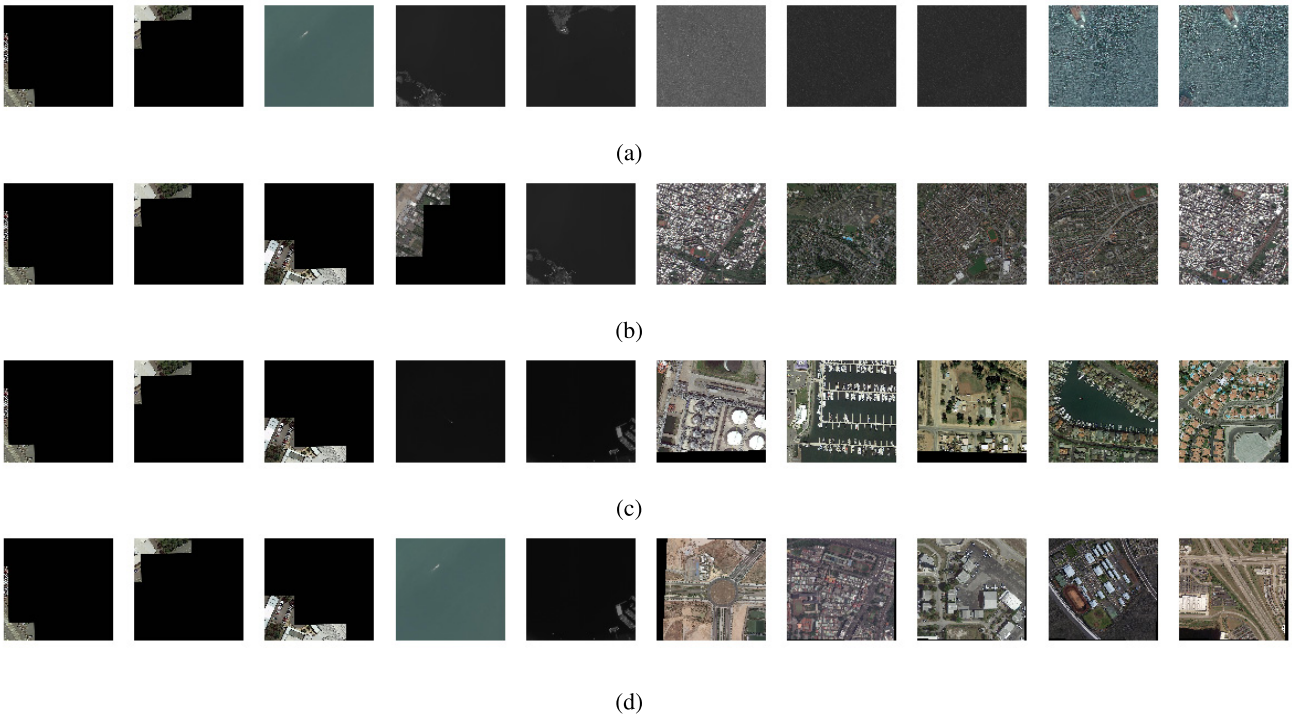


Fig. 4. Multiscale saliency analysis. The first five images in each row have the smallest S_I values from the C2 (row a) to C5 (row d) layer of ResNet50. In comparison, the last five images in each row have the largest S_I values from the same layer, respectively. The first five images in each group are visually simple and contain large portions of uniform background, whereas the last five images are visually complex with many objects and varying land features. Images with large S_I values obtained from earlier layers of ResNet50 (C2 and C3) have dense low-level image features (e.g., many tiny objects), whereas those from the later layers of ResNet50 (C4 and C5) have higher level image features (e.g., larger objects). Sample images with the smallest (first 5) and largest (last 5) S_I values from (a) C2, (b) C3, (c) C4, and (d) C5 layers of ResNet50.

on the horizontal bounding box (HBB) task in DOTA and set nonmaximum suppression (NMS) to 0.3 for all categories. The IoU ratio of predicted and ground-truth boxes is 0.5, as commonly used in the object detection domain.

For NWPU VHR-10 data, the parameter setting of performance evaluation is the same as the original article [27]. We used public tools (https://github.com/Cartucho/mAP) to calculate the mAP score. In order to show the robustness of the proposed ASBL-RetinaNet method, an ablation study was done on the DOTA data set and all hyperparameters were fixed for experiments on the NWPU VHR-10 data set.

C. RetinaNet Modification

In addition to reporting the performance of the original RetinaNet, we also made some minor changes to RetinaNet and achieved improved performance. Specifically, we changed the aspect ratios of the anchor boxes to {1:3, 1:1, 3:1}, and the sizes to $\{2^1, 2^{1/2}, 0.3\}$. The reasons for these changes are that some object categories, such as bridge or harbor, have long rectangle shapes, and some objects are very small, requiring small anchor size 0.3. In terms of data augmentation, instead of random flip used by the original RetinaNet, both random flip and flop were used.

D. Ablation Study

1) Image-Based ASBL Experiments and Analysis:

a) Image complexity analysis: In our method, the amount of activation in certain layers of a DNN is used to represent the complexity of an input image in ASBL$_I$. When the background of an image causes more neurons to be activated, the image is more complex. Fig. 4 shows some examples of DOTA images selected based on their S_I values. The images in each row are selected based on their S_I values computed based on their feature maps from C2 to C5 layers of ResNet50, respectively. The first five images in each group have the smallest S_I values and are visually simple and contain large portions of uniform background, whereas the last five images have the largest S_I values and are visually complex with many objects and varying land features. Images with large S_I values obtained from the earlier layers of ResNet50 (C2 and C3) have dense low-level image features (e.g., many tiny objects), whereas those from the later layers of ResNet50 (C4 and C5) have higher level image features (e.g., larger objects). The reason is that the receptive fields of C2 and C3 layers are smaller than those of the C4 and C5 layers. The feature maps generated from the C2 and C3 layers capture more small objects' information, but those from the C4 and C5 layers capture more information of larger objects. These examples show that S_I could represent the complexity of an input image at different feature levels quite well.

b) Saliency normalization: Fig. 5 shows the distribution of saliency values S_I of DOTA training images obtained from the residual block C2 and C5 layers of ResNet50, respectively. The S_I values from some layers, such as C5, have small ranges, which do not separate easy and hard cases sufficiently. Saliency normalization, as shown in (5), is proposed in our method to address this problem and control the value range of weighting factors in the overall loss function.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

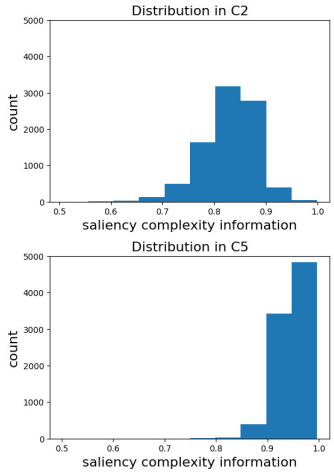


Fig. 5. Distribution of saliency values of DOTA training images obtained from residual block C2 and C5, respectively, of ResNet50.

TABLE I

TEST PERFORMANCES OF RETINANET ON THE DOTA DATA SET TRAINED BY ASBL-RETINANET USING THE NEW ASBL LOSS FUNCTION WITH OR WITHOUT SALIENCY NORMALIZATION IN (5)

S_{lb}	Normalization	mAP
0.3	Y	61.79
0.5	Y	**63.48**
0.7	Y	62.4
-	N	62.86

To show the effectiveness of the new loss functions with saliency information S_I, we compared the performances of the RetinaNet model trained using the new loss functions with those trained using the original loss function.

Table I shows some experimental results of ASBL-RetinaNet using the new ASBL loss functions with or without saliency normalization. The saliency values were calculated from the C5 layer of ResNet50. Among these results, the best performance was achieved when saliency normalization was used with S_{lb} 0.5, which was 0.62 mAP higher (63.48 versus 62.86) than the result without normalization.

In comparison, the mAP of the modified RetinaNet trained using the original loss function was 62.51, almost 1 mAP lower than the result using the new loss function (63.48). We ran these experiments multiple times and the mAP standard deviations for these two methods are 0.017 and 0.022, respectively, meaning their performance difference is significant.

In the ablation study of saliency normalization, we noticed that there were some optimal weights on the easy and hard cases in the loss functions. When the weights on easy cases were too small, the trained models would not perform well on easy cases and obtained lower overall performance. On the other hand, when the weights on easy cases were too large, the hard cases would not have much impact on training, and the new ASBL loss function has no advantage over the original RetinaNet loss function. The result is that the trained models would perform similar to the original RetinaNet.

c) Comparison of saliency information at different feature levels: Saliency values calculated based on features at different levels could lead to different model performances. Table II shows the test performance of RetinaNet trained using the new ASBL loss functions with saliency values calculated based on feature maps from the C2 to C5 layers, respectively, of ResNet50 with saliency normalization. The results show that the best performance (64.77 mAP) was achieved when using saliency values calculated from the C2 layer, which was 2.26 mAP higher than the original RetinaNet (62.51 mAP).

TABLE II

TEST PERFORMANCE OF RETINANET TRAINED USING THE NEW ASBL LOSS FUNCTIONS WITH SALIENCY VALUES CALCULATED BASED ON FEATURE MAPS FROM THE C2 TO C5 LAYERS, RESPECTIVELY, OF RESNET50

Conv Block Layer of ResNet50	C2	C3	C4	C5
mAP	**64.77**	64.51	63.32	63.48

TABLE III

TEST PERFORMANCE COMPARISON OF RESNET50 TRAINED USING IMAGE-BASED ASBL, ASBL$_I$, AND ANCHOR-BASED ASBL, ASBL$_A$, WITH FIXED OR DYNAMIC UPDATE OF ANCHOR SALIENCY, AND, OPTIONALLY, SALIENCY NORMALIZATION, ON THE DOTA DATA SET

Method	Dynamic update	Normalization	S_{lb}	mAP
$ASBL_A$	Y	Y	0.3	65.54
			0.5	**66.12**
			0.7	65.58
		N	-	65.53
	N	Y	0.5	64.82
$ASBL_I$	-	Y	0.5	64.77

2) Anchor Based ASBL Experiments and Analysis: Similar to the image-based ASBL, saliency values computed to represent the complexity of anchors can also be normalized in the anchor-based ASBL. For anchor saliency, we use the same normalization formula as for image-based saliency in (5). For each anchor, we use the maximum and minimum values of each feature map as (max) and (min) in the formula. For anchor-based ASBL, the anchor saliency values could be calculated in two ways. One is to calculate the anchor saliency values once in the beginning of training based on the feature maps generated by RetinaNet trained with ASBL$_I$, and fix them throughout the training phase using ASBL$_A$. The other is to recalculate the anchor saliency values during training based on updated RetinaNet weights, that is, dynamically updating anchor saliency values during training.

Table III shows the test performance comparison of ResNet50 trained using image-based ASBL, ASBL$_I$, and anchor-based ASBL, ASBL$_A$, with fixed or dynamic update of anchor saliency, and, optionally, saliency normalization. Saliency normalization was done with different S_{lb} (0.3, 0.5, and 0.7). The experimental results show that ASBL$_A$ using dynamic anchor saliency and saliency normalization can improve the performance of ASBL$_A$ using fixed anchor saliency from 64.82 to 66.12 mAP. The reason is that the features of objects are computed much more accurately during training and more accurate weighting value for each anchor is obtained. The best result of ASBL$_A$ was achieved with $S_{\text{lb}} = 0.5$. ASBL$_A$ performed better than ASBL$_I$. The best result of ASBL$_A$ in Table III is 66.12, significantly higher than using ASBL$_I$ (64.77) on the DOTA data set.

E. Experiment Setup

In our experiments, the DNN models trained by ASBL used ResNet50 [3] as the backbone of RetinaNet [18].

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
TEST PERFORMANCE OF VARIOUS OBJECT DETECTORS ON THE DOTA TEST DATA SET IN MAP

	Data	Plane	BD	Bridge	GTF	SV	LV	Ship	TC	BC	ST	SBF	RA	Harbor	SP	HC	mAP
YOLO [8]	T+V	76.9	33.87	22.73	34.88	38.73	32.02	52.37	61.65	48.54	33.91	29.27	36.83	36.44	38.26	11.61	39.2
SSD [11]	T+V	44.74	11.21	6.22	6.91	2	10.24	11.34	15.59	12.56	17.94	14.73	4.55	4.55	0.53	1.01	10.94
RFCN [9]	T+V	79.33	44.26	36.58	53.53	39.38	34.15	47.29	45.66	47.74	65.84	37.92	44.23	47.23	50.64	34.9	47.24
Faster RCNN [5]	T+V	80.32	**77.55**	32.86	**68.13**	53.66	52.49	50.04	90.41	**75.05**	59.59	**57**	49.81	61.69	56.46	**41.85**	60.46
RetinaNet [18]	T	78.22	53.41	26.38	42.27	63.64	52.63	73.19	87.17	44.64	57.99	18.03	51	43.39	56.56	7.44	50.39
RetinaNet*	T	89.03	62.14	43.88	47.05	73.57	65.18	78.65	90.86	66.28	70.26	35.07	58.26	68.93	66.34	22.16	62.51
ASBL-RetinaNet	T	89.09	67.96	46.38	57.12	73.55	66.19	**78.67**	90.86	71	**73.88**	45.15	60.92	70.01	68.51	32.49	66.12
ASBL-RetinaNet	T+V	**89.51**	74.07	**46.91**	55.54	**73.78**	**66.87**	78.48	**90.86**	70.09	73.2	46.71	**61.34**	**70.5**	**72.17**	32.84	**66.86**

* RetinaNet with modified anchor sizes and ratios

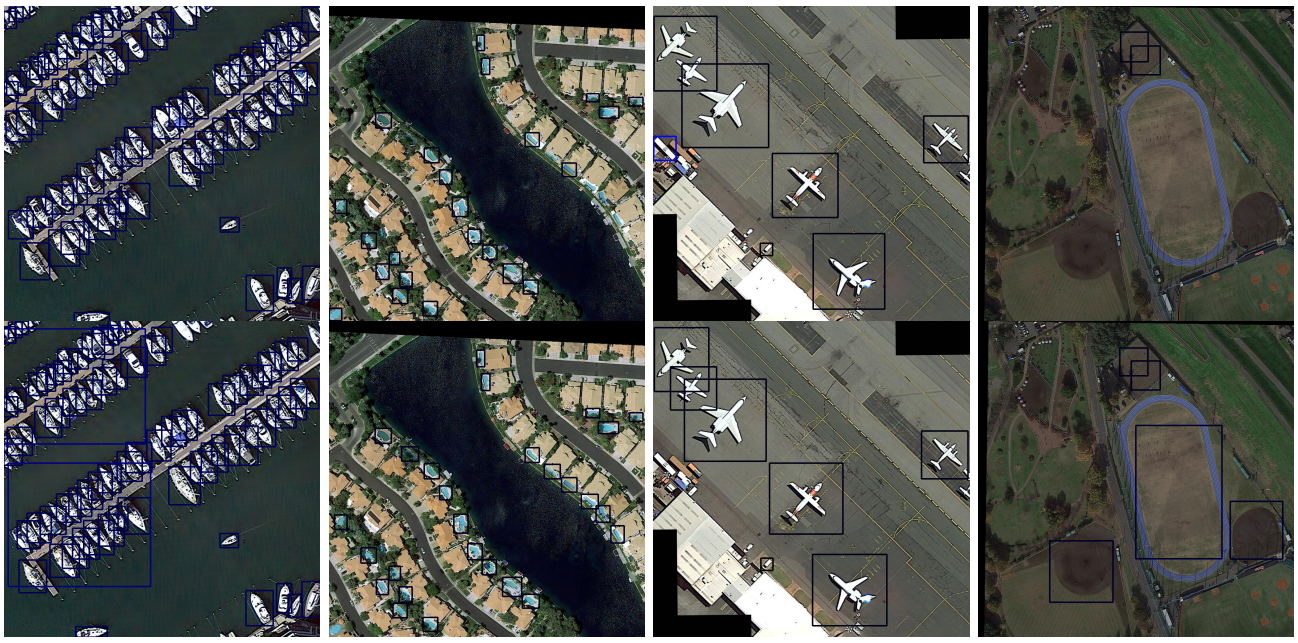


Fig. 6. Visual comparison of sample test results between the modified RetinaNet and ASBL-RetinaNet (threshold = 0.5). The top images are outputs of the modified RetinaNet, and the bottom ones are those of ASBL-RetinaNet. The first three columns show the improvement on objects of different scales, some crowded with complex backgrounds using the proposed ASBL. The fourth column shows the improvement on large objects using ASBL.

The input image size was 1024 * 1024 for DOTA images and 400 * 400 for NWPU VHR-10 images. For NWPU VHR-10, we resized the images to 600 * 600, to match the input image size of RetinaNet in its original article. In training, we used Adam as the solver with beta (0.9, 0.999), eps (1e−8), and weight decay (0). One Titan X GPU desktop was used in the experiments with training batch size 2. ResNet50 pretrained on ImageNet was used as the initial backbone model. Random flip and flop were used for data augmentation. Unless otherwise specified, all models were trained for 50 epochs with initial learning rate 0.0001, which was then divided by 10 after every 20 epochs. During training, for the ASBL method, we first used the image-based ASBL loss function to train the network for 25 epochs, and then used the anchor-based loss function to train the network for 25 more epochs. The training data sets of DOTA and NWPU VHR-10, the same as in their original published articles, were used in training.

F. Experimental Results on DOTA

On the largest publicly available aerial image data set, DOTA, we compared the performances of RetinaNet trained using the new ASBL loss functions with some recent state-of-the-art deep learning methods, including both one-stage and two-stage detectors.

Table IV shows the performances of various existing object detectors, including YOLO, SSD, RFCN, Faster RCNN, RetinaNet, as well as ASBL-RetinaNet, on the test data set. In the "Data" column, "T" means that a model was trained using the training data set of DOTA, whereas "T+V" means that a model was trained using the combined training and validation data set. Their performances on each target categories, as well as the overall average in mAP (the last column), are shown.

The results show that the new method ASBL-RetinaNet trained using T+V achieved the highest average precision, 66.86, which was 6.4 mAP higher than the best previously published result, Faster RCNN [5] (60.46). We implemented the modified RetinaNet with different anchor sizes and ratios from the original RetinaNet and got much better performance than the original RetinaNet. Across all 15 target categories, ASBL-RetinaNet outperformed Faster RCNN in ten categories and modified RetinaNet in all 15 categories using DOTA training data only. ASBL-RetinaNet outperformed the modified

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

TABLE V
TEST PERFORMANCES OF VARIOUS OBJECT DETECTORS ON THE NWPU VHR-10 TEST DATA SET IN MAP

	Airplane	Ship	Storage tank	Baseball diamond	Tennis court	Basketball court	Ground track field	Harbor	Bridge	Vehicle	mAP
COPD [42]	62.25	69.37	64.52	82.13	34.13	35.25	84.21	56.31	16.43	44.28	54.89
Transferred CNN [1]	66.03	57.13	85.01	80.93	35.11	45.52	79.37	62.57	43.17	41.27	59.61
RICNN [30]	88.71	78.34	86.33	89.09	42.33	56.85	87.72	67.47	62.31	72.01	73.11
Faster RCNN [5]	90.9	86.3	90.53	**98.24**	89.72	69.64	100	80.11	61.49	78.14	84.51
Li etc [27]	99.70	90.8	90.61	92.91	**90.29**	80.13	90.81	80.29	68.53	**87.14**	87.12
RetinaNet* [18]	96.58	83.61	74.76	84.32	63.99	59.66	98.33	62.83	65.72	78.31	76.81
ASBL-RetinaNet (VGG16)	**100**	91.27	**96.76**	96.69	68.54	**87.67**	100	77.10	**83.92**	83.45	88.54
ASBL-RetinaNet (ResNet50)	99.34	**93.19**	94.36	97.70	71.19	84.67	**100**	**87.61**	81.49	83.50	**89.31**

RetinaNet by 3.61 mAP (66.12 versus 62.51) when trained using the DOTA training data set only. The inference speeds of the various RetinaNet models are the same, since their neural network architectures are the same and they differ in the network weights. Compared to all other methods, ASBL-RetinaNet was the best for 8 out of 15 target categories.

Fig. 6 compares the detection results of the modified RetinaNet (top) and our proposed ASBL-RetinaNet (bottom) on four examples of DOTA test images. Between the two results in the first column, the modified RetinaNet misclassified the harbors as background due to the crowded boats; Yet ASBL-RetinaNet detected the harbors while detecting the boats equally well. As shown in the results in the second and third columns in Fig. 6, RetinaNet missed some objects of different scales, like some SPs on the lower right side of the river and an airplane in the top left corner, while ASBL-RetinaNet detected them successfully. Even though RetinaNet is designed to solve the multiscale problem using FPN as feature extraction, the images in second and third columns that include complex background and crowded objects still have a problem. The main reason is that RetinaNet did not consider those cases. In ASBL-RetinaNet, we solve the complex background problem using image-based ASBL, and crowded objects problem, which may have complex information in each anchor, using anchor-based ASBL. The images in second and third columns were given higher weights in training, which led to improved detection accuracy on objects of different scales by image-based ASBL and anchor-based ASBL, respectively. In the image-based ASBL, the more complex input images, like the one in the second column, are assigned higher weights so that the training error would be larger if the result misses a multiscale object. Anchor-based ASBL improves performance on images containing crowded objects, like the one in the third column, with more training effort on crowded objects which has more complex information for each anchor of those objects. In addition, the fourth column shows that ASBL-RetinaNet can improve performance on images with easy backgrounds, even though it gave more weights to images with complex backgrounds in training.

G. Experimental Results on NWPU VHR-10

Table V compares the performance of ASBL-RetinaNet with various existing deep learning methods, including collection of part detectors (COPD) [42], Transferred CNN [1], rotation-invariant convolution neural network (RICNN) [30], Faster RCNN [5], Li's method [27], and the modified RetinaNet on the test data set of NWPU VHR-10. ASBL-RetinaNet with ResNet50 as the backbone was the best, achieving 89.31, 2.19 mAP higher than the best previous result, Li's method, (87.12 mAP) and 12.5 mAP higher than the modified RetinaNet. Since Li's method and Faster RCNN use VGG-16 as their backbone, we also implemented ASBL-RetinaNet with VGG-16 as the backbone and its result was better than both Li's method and Faster RCNN, outperforming Li's method by 1.42 mAP. Compared to RetinaNet trained using the original loss function, RetinaNet trained using the new ASBL loss functions was better on all target categories significantly.

TABLE VI
INFERENCE TIME COMPARISON BETWEEN VARIOUS OBJECT DETECTORS ON NWPU VHR-10 TEST IMAGES

Method	Average running time per image (sec)
COPD [42]	1.16
Transferred CNN [1]	5.09
RICNN [30]	8.47
Faster RCNN [5]	0.09
Li etc [27]	2.89
RetinaNet	**0.045**
ASBL-RetinaNet	**0.045**

Compared to existing methods, ASBL-RetinaNet achieved bigger performance improvement on NWPU VHR-10 images than DOTA images. There are mainly two reasons.

1) NWPU VHR-10 images are more complex than DOTA images. The average C2 saliency of NWPU VHR-10 images is 0.88, which is larger than 0.84, the average C2 saliency of DOTA images. ASBL works better on complex images.
2) NWPU VHR-10 has 293 test images, much fewer than DOTA with more than 1000 test images. The performance variance on NWPU VHR-10 test images is larger than that on DOTA test images.

Table VI shows the inference speed comparison between various methods. ASBL-RetinaNet was two times faster than Faster RCNN, which is the fastest among previous methods. It took 45 ms per image using an NVIDIA Titan X GPU and 16 GB of memory as reported in their article [27]. In our experiments, we used an NVIDIA Titan X GPU and 12 GB of memory.

Fig. 7 shows the detection results of ASBL-RetinaNet on six sample images from the NWPU VHR-10 test data set.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.



Fig. 7. Detection results of ASBL-RetinaNet on six sample images from the NWPU VHR-10 test data set.

The method successfully detected objects of various sizes and shapes in these images.

V. Conclusion

In this article, we proposed a new method for designing loss function, called ASBL, for training existing DNN object detectors on images. ASBL can be implemented at the image level, which is called image-based ASBL, and at the anchor level, which is called anchor-based ASBL. They give training images of different weights based on image complexity. Without loss of generality, the ASBL method was applied to RetinaNet, one of the best existing one-stage object detectors, to show its effectiveness. Using two large benchmark data sets, DOTA and NWPU VHR-10, experimental results show that ASBL-RetinaNet outperformed existing state-of-the-art deep learning methods, with at least 6.4-mAP improvement on DOTA, and 2.19 mAP on NWPU VHR-10. Furthermore, ASBL-RetinaNet improved over the original RetinaNet by 3.61 mAP on DOTA images and 12.5 mAP on NWPU VHR-10 images. In the future work, rotation and scale information of objects in training images could also be included in the loss function for further improvement.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*. [Online]. Available: http://arxiv.org/abs/1409.1556

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[4] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 580–587.

[5] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 91–99.

[6] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.

[7] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, vol. 1, no. 2, p. 3.

[8] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," 2016, *arXiv:1612.08242*. [Online]. Available: http://arxiv.org/abs/1612.08242

[9] J. Dai, Y. Li, K. He, and J. Sun, "R-FCN: Object detection via region-based fully convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 379–387.

[10] A. Shrivastava, A. Gupta, and R. Girshick, "Training region-based object detectors with online hard example mining," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 761–769.

[11] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Computer Vision—ECCV* (Lecture Notes in Computer Science), vol. 9905, B. Leibe, J. Matas, N. Sebe, M. Welling, Eds. Cham, Switzerland: Springer, 2016, pp. 21–37.

[12] G. Chen, P. Sun, and Y. Shang, "Automatic fish classification system using deep learning," in *Proc. IEEE 29th Int. Conf. Tools Artif. Intell. (ICTAI)*, Nov. 2017, pp. 24–29.

[13] T. Tang, S. Zhou, Z. Deng, H. Zou, and L. Lei, "Vehicle detection in aerial images based on region convolutional neural networks and hard negative example mining," *Sensors*, vol. 17, no. 2, p. 336, 2017.

[14] L. W. Sommer, T. Schuchert, and J. Beyerer, "Deep learning based multi-category object detection in aerial images," in *Proc. Autom. Target Recognit.*, May 2017, Art. no. 1020209.

[15] X. Yang *et al.*, "Automatic ship detection in remote sensing images from Google Earth of complex scenes based on multiscale rotation dense feature pyramid networks," *Remote Sens.*, vol. 10, no. 1, p. 132, 2018.

[16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.

[17] T. Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Computer Vision—ECCV* (Lecture Notes in Computer Science), vol. 8693, D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars, Eds. Cham, Switzerland: Springer, 2014, pp. 740–755.

[18] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020.

[19] P. Sun, N. M. Wergeles, C. Zhang, L. M. Guerdan, T. Trull, and Y. Shang, "ADA—Automatic detection of alcohol usage for mobile ambulatory assessment," in *Proc. IEEE Int. Conf. Smart Comput. (SMARTCOMP)*, May 2016, pp. 1–5.

[20] J. P. Bernstein, B. J. Mendez, P. Sun, Y. Liu, and Y. Shang, "Using deep learning for alcohol consumption recognition," in *Proc. 14th IEEE Annu. Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2017, pp. 1020–1021.

[21] G.-S. Xia *et al.*, "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3974–3983.

[22] S. Li, Z. Zhang, B. Li, and C. Li, "Multiscale rotated bounding box-based deep learning method for detecting ship targets in remote sensing images," *Sensors*, vol. 18, no. 8, p. 2702, 2018.

[23] S. Majid Azimi, E. Vig, R. Bahmanyar, M. Körner, and P. Reinartz, "Towards multi-class object detection in unconstrained remote sensing imagery," 2018, *arXiv:1807.02700*. [Online]. Available: http://arxiv.org/abs/1807.02700

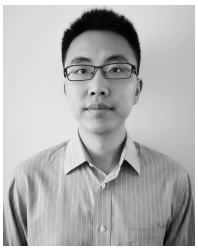
[24] D. Zhang, D. Meng, and J. Han, "Co-saliency detection via a self-paced multiple-instance learning framework," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 5, pp. 865–878, May 2017.

[25] D. Zhang, J. Han, C. Li, J. Wang, and X. Li, "Detection of co-salient objects by looking deep and wide," *Int. J. Comput. Vis.*, vol. 120, no. 2, pp. 215–232, Nov. 2016.

[26] A. Recasens, P. Kellnhofer, S. Stent, W. Matusik, and A. Torralba, "Learning to zoom: A saliency-based sampling layer for neural networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 51–66.

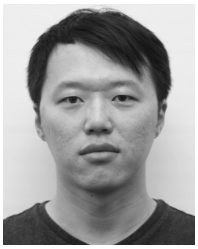
Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.

[27] K. Li, G. Cheng, S. Bu, and X. You, "Rotation-insensitive and context-augmented object detection in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 2337–2348, Apr. 2018.
[28] P. Sun, G. Chen, G. Luke, and Y. Shang, "Salience biased loss for object detection in aerial images," 2018, *arXiv:1810.08103*. [Online]. Available: http://arxiv.org/abs/1810.08103
[29] G. Cheng and J. Han, "A survey on object detection in optical remote sensing images," *ISPRS J. Photogram. Remote Sens.*, vol. 117, pp. 11–28, Jul. 2016.
[30] G. Cheng, P. Zhou, and J. Han, "Learning rotation-invariant convolutional neural networks for object detection in VHR optical remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 12, pp. 7405–7415, Dec. 2016.
[31] G. Cheng, J. Han, P. Zhou, and D. Xu, "Learning rotation-invariant and Fisher discriminative convolutional neural networks for object detection," *IEEE Trans. Image Process.*, vol. 28, no. 1, pp. 265–278, Jan. 2019.
[32] J. Han, D. Zhang, G. Cheng, L. Guo, and J. Ren, "Object detection in optical remote sensing images based on weakly supervised learning and high-level feature learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 6, pp. 3325–3337, Jun. 2015.
[33] F. Provost, "Machine learning from imbalanced data sets 101," in *Proc. AAAI Workshop Imbalanced Data Sets*, vol. 68. AAAI Press, 2000.
[34] Q. Dong, S. Gong, and X. Zhu, "Class rectification hard mining for imbalanced deep learning," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1851–1860.
[35] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1440–1448.
[36] C.-Y. Fu, W. Liu, A. Ranga, A. Tyagi, and A. C. Berg, "DSSD: Deconvolutional single shot detector," 2017, *arXiv:1701.06659*. [Online]. Available: http://arxiv.org/abs/1701.06659
[37] C. L. Zitnick and P. Dollár, "Edge boxes: Locating object proposals from edges," in *Computer Vision—ECCV* (Lecture Notes in Computer Science), vol. 8693, D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars, Eds. Cham, Switzerland: Springer, 2014, pp. 391–405.
[38] S. Yang, P. Luo, C. C. Loy, and X. Tang, "WIDER FACE: A face detection benchmark," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 5525–5533.
[39] R. T. Ionescu, B. Alexe, M. Leordeanu, M. Popescu, D. P. Papadopoulos, and V. Ferrari, "How hard can it be? Estimating the difficulty of visual search in an image," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2157–2166.
[40] Q. Tao, H. Yang, and J. Cai, "Exploiting web images for weakly supervised object detection," *IEEE Trans. Multimedia*, vol. 21, no. 5, pp. 1135–1146, May 2019.
[41] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Jun. 2010.
[42] G. Cheng, J. Han, P. Zhou, and L. Guo, "Multi-class geospatial object detection and geographic image classification based on collection of part detectors," *ISPRS J. Photogram. Remote Sens.*, vol. 98, pp. 119–132, Dec. 2014.



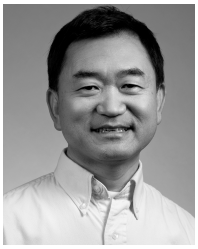
Peng Sun received the M.A. degree in statistics and the Ph.D. degree in computer science from the University of Missouri, Columbia, MO, USA, in 2014 and 2019, respectively.

He is an artificial intelligence (AI) Scientist with The Climate Corporation, San Francisco, CA, USA. He works on applying deep learning and AI algorithm into digital farming. His research interests include deep learning, object detection, and computer vision.



Guang Chen received the M.E. degree in electrical and computer engineering from the University of Missouri, Columbia, MO, USA, in 2015, where he is pursuing the Ph.D. degree in computer science.

He is a Research Scientist with the JD.COM Silicon Valley Research Center, Mountain View, CA, USA, where he also works on edge computing-based computer vision applications for smart farming. His research interests include object detection, object tracking, and computer vision.



Yi Shang (Senior Member, IEEE) received the B.S. degree from the University of Science and Technology of China, Hefei, China, in 1988, the M.S. degree from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 1991, and the Ph.D. degree in computer science from the University of Illinois at Urbana–Champaign, Champaign, IL, USA, in 1997.

He worked as a Researcher with the Xerox Palo Alto Research Center (PARC), Palo Alto, CA, USA. He is a Professor and the Director of graduate studies with the Department of Electrical Engineering and Computer Science, University of Missouri, Columbia, MO, USA. He has published over 200 refereed articles in the fields of artificial intelligence (AI), wireless sensor networks, mobile and distributed computing, and bioinformatics. He received six U.S. patents. His research has been supported by grants from many funding sources, including NSF, NIH, U.S. Army, DARPA, the Missouri Department of Conservation, Microsoft, Raytheon, and so on.

Dr. Shang is a Lifetime Member of Association for Computing Machinery (ACM).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:35:13 UTC from IEEE Xplore. Restrictions apply.